

FreeRTOS移植到STM32



本文详细介绍了如何将FreeRTOS实时操作系统移植到STM32F103裸机工程中，包括从官网下载源码、创建工程目录结构、添加源码到工程、配置FreeRTOSConfig.h文件以及修改中断服务函数等关键步骤。通过这个教程，读者可以学习到FreeRTOS在STM32上的移植过程。

一、找一个STM32的裸机工程模板

我们以STM32F103裸机程序为例



随便找的一个裸机程序

二、去官网下载FreeRTOS V9.0.0 源码

在移植之前，我们首先要获取到FreeRTOS的官方的源码包。这里我们提供两个下载链接，一个是官网：<http://www.freertos.org/>，另外一个代码托管网站：<https://sourceforge.net/projects/freertos/files/FreeRTOS/>。虽然不是最新版本的源码包但是因为内核很稳定，并且网上资料很多所以我们选用V9.0.0版本

我们打开FreeRTOS的代码托管网站，就可以看到FreeRTOS的源码及其版本信息了，具体见图

← × 🏠 https://sourceforge.net/projects/freertos/files/FreeRTOS/		
 SOURCEFORGE		Open Source Software Business Software
📁 v202012.00	2020-12-15	
📁 v202011.00	2020-12-15	
📁 V10.4.1	2020-09-22	
📁 V10.4.0	2020-09-17	
📁 V7.2.0	2020-02-10	
📁 V10.3.0	2020-02-07	
📁 V7.6.0	2020-02-04	
📁 V10.2.1	2019-12-02	
📁 V10.2.0	2019-02-25	
📁 Utils	2019-02-10	
📁 V8.2.3	2018-09-26	
📁 V8.2.1	2018-09-12	
📁 V10.1.1	2018-09-07	
📁 V9.0.0	2018-08-24	
📁 V10.1.0	2018-08-23	
📁 V10.0.1	2017-12-20	
📁 V10.0.0	2017-11-29	
📁 V8.2.2	2015-08-14	
📁 V8.2.0	2015-01-16	
📁 V8.2.0rc1	2014-12-24	
📁 V8.1.2	2014-09-02	

点开**V9.0.0**下载zip这个，

Download Latest Version
FreeRTOSv20212.00-LTS.exe (31.7 MB)

Get Updates

Home / FreeRTOS / V9.0.0

Name	Modified	Size	Downloads / Week
Parent folder			
FreeRTOS_V9.0.0a (V9 with MIT).zip	2018-08-24	1.4 MB	22
source-code-for-book-examples.zip	2016-12-18	439.3 kB	303
FreeRTOSv9.0.0.zip	2016-05-25	51.4 MB	79
README	2016-05-25	121.5 kB	6
FreeRTOSv9.0.0.exe	2016-05-25	16.1 MB	11
Totals: 5 Items		69.5 MB	421

Documentation and download available at <http://www.FreeRTOS.org/>

Changes between FreeRTOS V9.0.0 and FreeRTOS V9.0.0rc2 released May 25 2016:

解压完成后就会得到一个完整的Freertos源码包

FreeRTOSv9.0.0

文件 主页 共享 查看

固定到快速访问 复制 粘贴 剪贴板

移动到 复制到 删除 重命名

新建项目 新建文件夹 轻松访问

打开 属性 编辑 历史记录

全部选择 全部取消 反向选择

此电脑 新加卷 (E:) freertos 111 FreeRTOSv9.0.0

名称

修改日期

类型

大小

FreeRTOS

2016/5/20 20:25

文件夹

FreeRTOS-Plus

2016/5/20 20:20

文件夹

New - Direct to Task Notifications

2015/1/16 18:56

Internet 快捷方式

1 KB

New - FreeRTOS+TCP

2014/12/22 3:03

Internet 快捷方式

1 KB

Quick_Start_Guide

2013/9/17 16:17

Internet 快捷方式

1 KB

readme.txt

2013/9/17 16:20

文本文档

2 KB

Upgrading-to-FreeRTOS-9

2016/2/19 17:32

Internet 快捷方式

1 KB

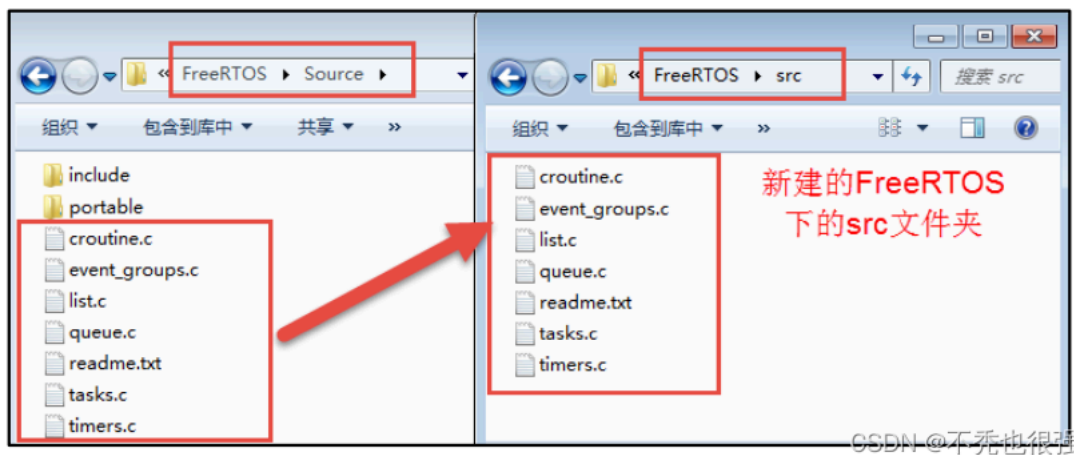
在 FreeRTOSv9.0.0 中搜索

FreeRTOS 包含 Demo 例程和内核源码（比较重要，我们就需要提取该目录下的大部分文件），具体见图 13-5。FreeRTOS 文件夹下的 Source 文件夹里面包含的是 FreeRTOS 内核的源代码，我们移植 FreeRTOS 的时候就需要这部分源代码；FreeRTOS 文件夹下的 Demo 文件夹里面包含了 FreeRTOS 官方为各个单片机移植好的工程代码，FreeRTOS 为了推广自己，会给各种半导体厂商的评估板写好完整的工程程序，这些程序就放在 Demo 这个目录下，这部分 Demo 非常有参考价值。我们把 FreeRTOS 到 STM32 的时候，FreeRTOSConfig.h 这个头文件就是从这里拷贝过来的，我们这里不做详解

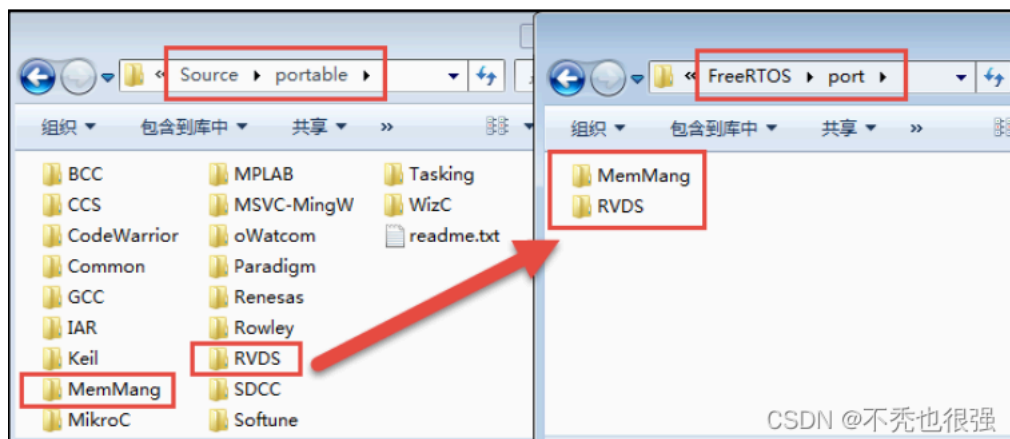
三、往裸机工程添加 FreeRTOS 源码

1. 首先在我们的 STM32 裸机工程模板根目录下新建一个文件夹，命名为“FreeRTOS”，并且在 FreeRTOS 文件夹下新建两个空文件夹，分别命名为“src”与“port”，src 文件夹用于保存 FreeRTOS 中的核心源文件，也就是我们常说的‘.c 文件’，port 文件夹用于保存内存管理以及处理器架构相关代码，这些代码 FreeRTOS 官方已经提供给我们的，直接使用即可，在前面已经说了，FreeRTOS 是软件，我们的开发版是硬件，软硬件必须有桥梁来连接，这些与处理器架构相关的代码，可以称之为 RTOS 硬件接口层，它们位于 FreeRTOS/Source/Portable 文件夹下。

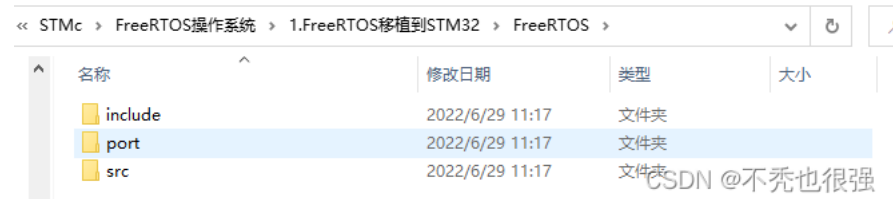
2. 打开 FreeRTOS V9.0.0 源码，在“FreeRTOSv9.0.0\FreeRTOS\Source”目录下找到所有的‘.c 文件’，将它们拷贝到我们新建的 src 文件夹中，



3. 打开 FreeRTOS V9.0.0 源码，在“FreeRTOSv9.0.0\FreeRTOS\Source\portable”目录下找到“MemMang”文件夹与“RVDS”文件夹，将它们拷贝到我们新建的 port 文件夹中，

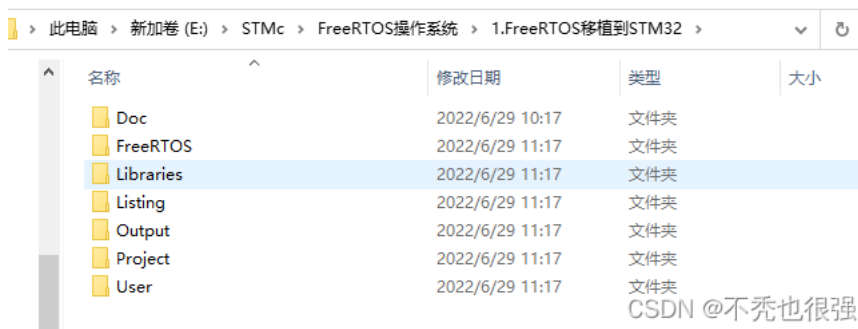


4. 打开 FreeRTOS V9.0.0 源码，在“FreeRTOSv9.0.0\FreeRTOS\Source”目录下找到“include”文件夹，它是我们需要用到 FreeRTOS 的一些头文件，将它直接拷贝到我们新建的 FreeRTOS 文件夹中，完成这一步之后就可以看到我们新建的 FreeRTOS 文件夹已经有 3 个文件夹，这 3 个文件夹就包含 FreeRTOS 的核心文件，至此，FreeRTOS 的源码就提取完成，



5. 拷贝 FreeRTOS 到裸机工程根目录

鉴于 FreeRTOS 容量很小，我们直接将刚刚提取的整个 FreeRTOS 文件夹拷贝到我们的 STM32 裸机工程里面，让整个 FreeRTOS 跟随我们的工程一起发布，使用这种方法打包的 FreeRTOS 工程，即使是将工程拷贝到一台没有安装 FreeRTOS 支持包（MDK 中有 FreeRTOS 的支持包）的电脑上面都是可以直接使用的，因为工程已经包含了 FreeRTOS 的源码。

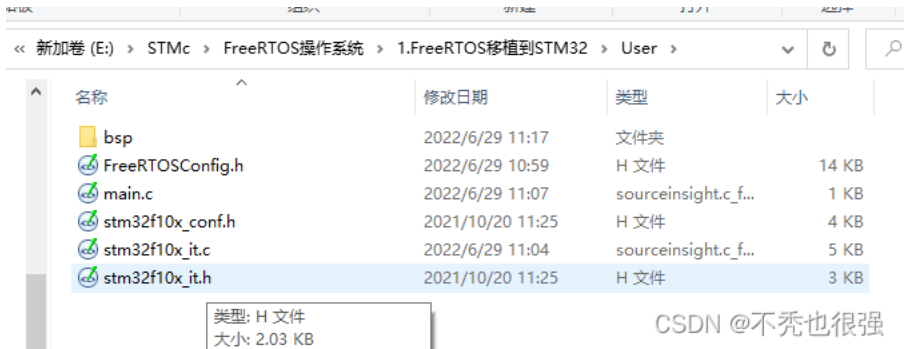


FreeRTOS 文件夹下就是我们提取的 FreeRTOS 的核心代码，该文件夹下的具体内容作用，这里就不再赘述。

6. 拷贝 FreeRTOSConfig.h 文件到 user 文件夹

FreeRTOSConfig.h 文件是 FreeRTOS 的工程配置文件，因为 FreeRTOS 是可以裁剪的实时操作内核，应用于不同的处理器平台，用户可以通过修改这个 FreeRTOS 内核的配置头文件来裁剪 FreeRTOS 的功能，所以我们把它拷贝一份放在 user 这个文件夹下面。打开 FreeRTOSv9.0.0 源码，

在“FreeRTOSv9.0.0\FreeRTOS\Demo”文件夹下面找到“CORTEX_STM32F103_Keil”这个文件夹，双击打开，在其根目录下找到这个“FreeRTOSConfig.h”文件，然后拷贝到我们工程的用户文件夹下即可，

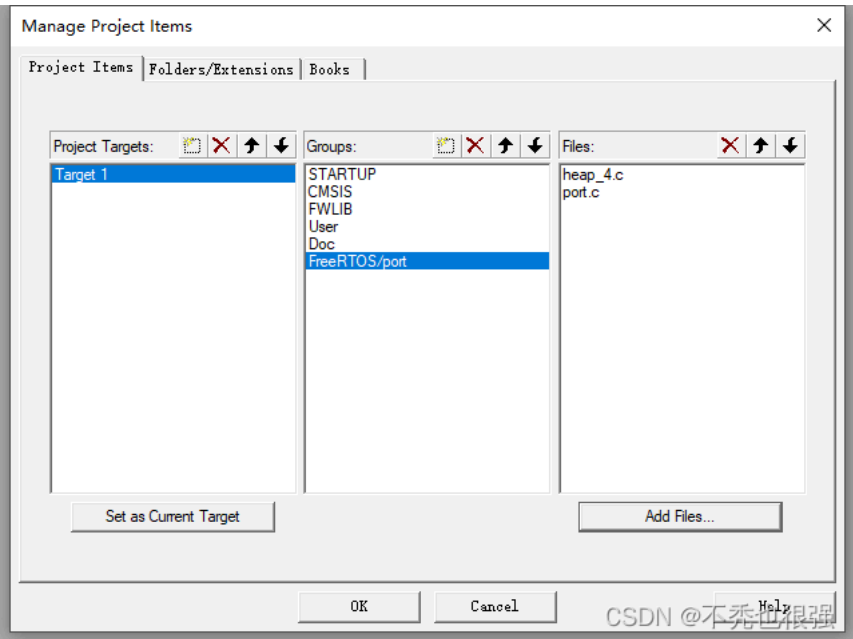


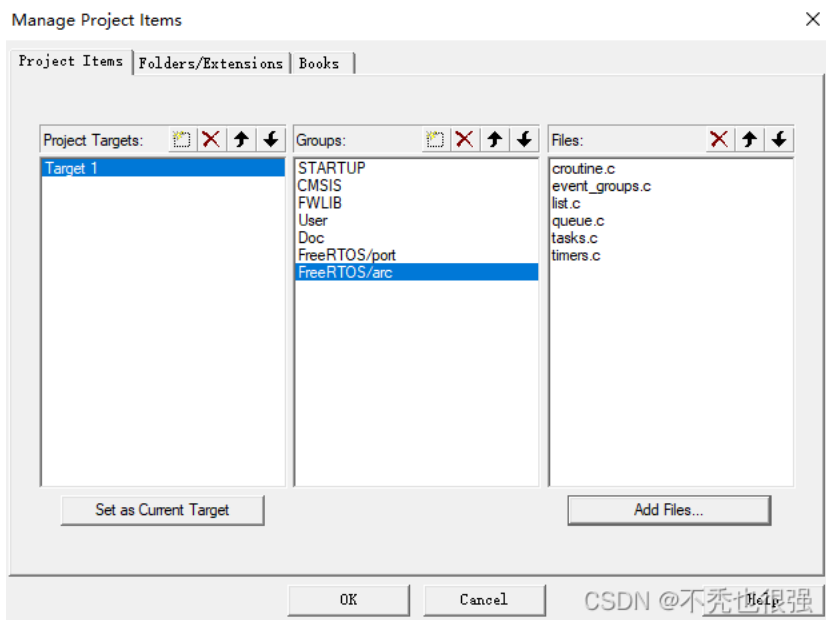
7.添加 FreeRTOS 源码到工程组文件夹

在上一步我们只是将 FreeRTOS 的源码放到了本地工程目录下，还没有添加到开发环境里面的组文件夹里面，FreeRTOS 也就没有移植到我们的工程中去。

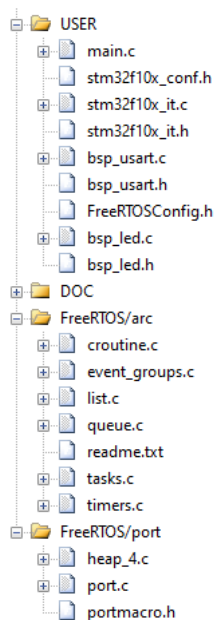
新建 FreeRTOS/src 和 FreeRTOS/port 组，接下来我们在开发环境里面新建 FreeRTOS/src 和 FreeRTOS/port 两个组文件夹，其中 FreeRTOS/src 用于存放 src 文件夹的内容，FreeRTOS/port 用于存放 port\MemMang 文件夹与 port\RVDS\ARM_CM? 文件夹的内容，“?”表示 3、4 或者 7，具体选择哪个得看你使用的是哪个型号的 STM32 开发板，如果是F3系列就拷贝port\RVDS\ARM_CM3

F4系列就拷贝port\RVDS\ARM_CM4，然后将工程文件中 FreeRTOS 的内容添加到工程中去，按照已经新建的分组添加 我们的 FreeRTOS 工程源码。在 FreeRTOS/port 分组中添加 MemMang 文件夹中的文件只需选择其中一个即可，我们选择“heap_4.c”，这是 FreeRTOS 的一个内存管理源码文件。同时，需要根据自己的开发板型号在 FreeRTOS\port\RVDS\ARM_CM?中选择，“?”表示 3、4 或者 7，具体选择 哪个得看你使用的是哪个型号的 STM32 开发板。

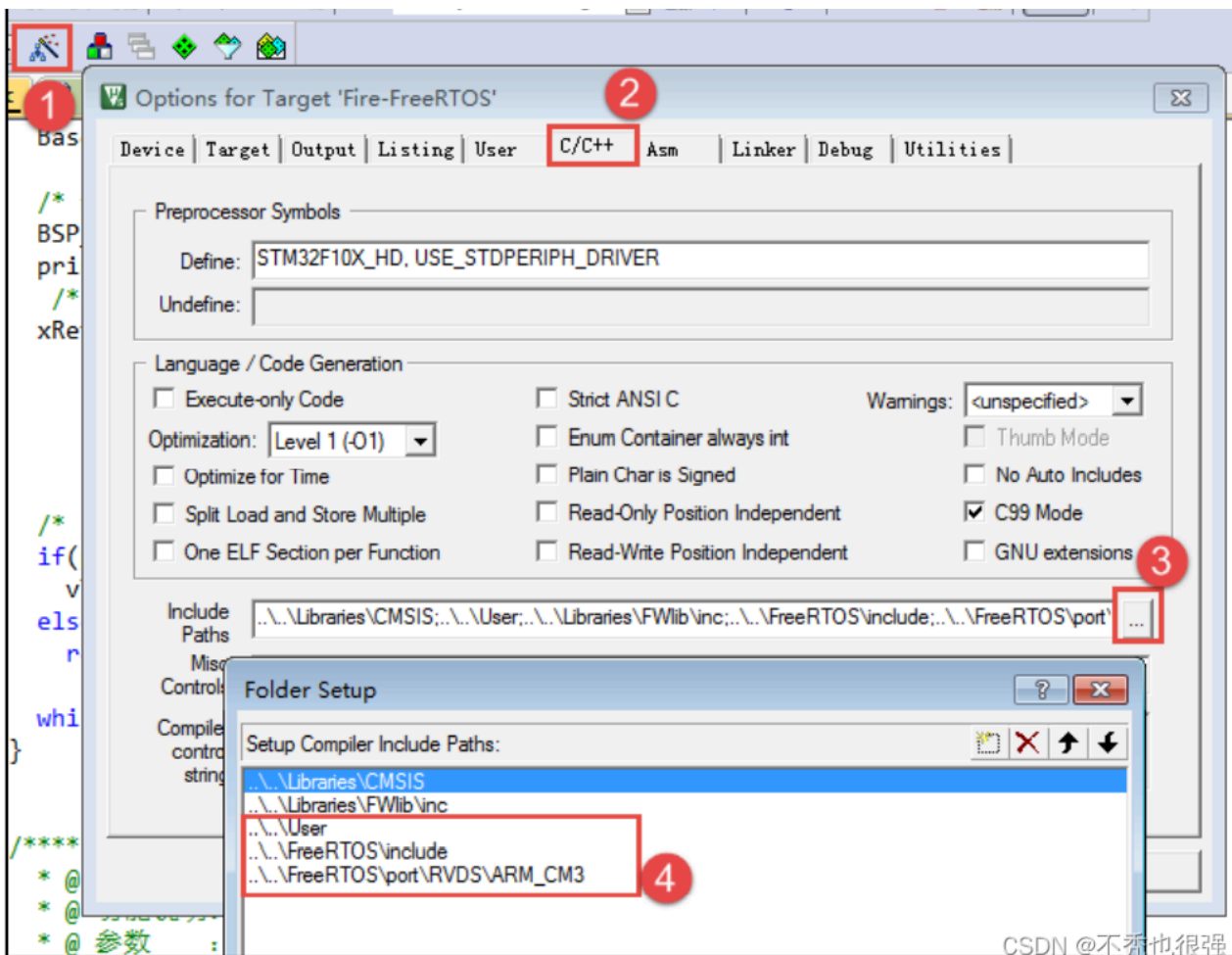




然后在 user 分组中添加我们 FreeRTOS 的配置文件 “FreeRTOSConfig.h”，因为这是 头文件（.h），所以需要在添加时选择文件类型为 “All files (*.*)”，至此我们的 FreeRTOS 添加到工程中就已经完成，效果图如下



8.指定 FreeRTOS 头文件的路径



这一步不过多解释，用过STM32的人都知道

9.FreeRTOSConfig.h 文件修改

FreeRTOSConfig.h 头文件的内容修改的不多，具体是：修改与对应开发板的头文件，如果是使用野火 STM32F1 的开发板，则包含 F1 的头文件 #include "stm32f10x.h"，同理是使用了其它系列的开发板，则包含与开发板对应的头文件即可，当然还需要包含我们的串口的头文件 "bsp_usart.h"，因为在我们 FreeRTOSConfig.h 中实现了断言操作，需要打印一些信息。其他根据需求修改即可，

```

#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

#include "stm32f10x.h"
#include "bsp_usart.h"

//针对不同的编译器调用不同的stdint.h文件
#if defined(__ICCARM__) || defined(__CC_ARM) || defined(__GNUC__)
#include <stdint.h>
extern uint32_t SystemCoreClock;
#endif

```

10. 修改 stm32f10x_it.

SysTick 中断服务函数是一个非常重要的函数，FreeRTOS 所有跟时间相关的事情都在里面处理，SysTick 就是 FreeRTOS 的一个心跳时钟，驱动着 FreeRTOS 的运行，就像人的心跳一样，假如没有心跳，我们就相当于“死了”，同样的，FreeRTOS 没有了心跳，那么它就会卡死在某个地方，不能进行任务调度，不能运行任何东西，因此我们需要实现一个 FreeRTOS 的心跳时钟，FreeRTOS 帮我们实现了 SysTick 的启动的配置：在 port.c 文件中已经实现 vPortSetupTimerInterrupt() 函数，并且 FreeRTOS 通用的 SysTick 中断服务函数也实现了：在 port.c 文件中已经实现 xPortSysTickHandler() 函数，所以移植的时候只需要我们在 stm32f10x_it.c 文件中实现我们对应（STM32）平台上的 SysTick_Handler() 函数即可。FreeRTOS 为开发者考虑得特别多，PendSV_Handler() 与 SVC_Handler() 这两个很重要的函数都帮我们实现了，在 port.c 文件中已经实现 xPortPendSVHandler() 与 vPortSVCHandler() 函数，防止我们自己实现不了，那么在 stm32f10x_it.c 中就需要我们注释掉 PendSV_Handler() 与 SVC_Handler() 这两个函数了，具体看一下代码

```

1. /* Includes -----*/
2. #include "stm32f10x_it.h"
3. #include "FreeRTOS.h" //FreeRTOS使用
4. #include "task.h"

```

```

5.  /** @addtogroup STM32F10x_StdPeriph_Template
6.      * @{
7.      */
8.
9.  /* Private typedef -----*/
10. /* Private define -----*/
11. /* Private macro -----*/
12. /* Private variables -----*/
13. /* Private function prototypes -----*/
14. /* Private functions -----*/
15.
16. /*****
17.  *      Cortex-M3 Processor Exceptions Handlers
18.  *****/
19.
20. /**
21.  * @brief This function handles NMI exception.
22.  * @param None
23.  * @retval None
24.  */
25. void NMI_Handler(void)
26. {
27. }
28.
29. /**
30.  * @brief This function handles Hard Fault exception.
31.  * @param None
32.  * @retval None
33.  */
34. void HardFault_Handler(void)
35. {
36.     /* Go to infinite loop when Hard Fault exception occurs */
37.     while (1)
38.     {
39.     }
40. }
41.
42. /**
43.  * @brief This function handles Memory Manage exception.
44.  * @param None
45.  * @retval None
46.  */
47. void MemManage_Handler(void)
48. {
49.     /* Go to infinite loop when Memory Manage exception occurs */
50.     while (1)
51.     {
52.     }
53. }
54.
55. /**
56.  * @brief This function handles Bus Fault exception.
57.  * @param None
58.  * @retval None
59.  */
60. void BusFault_Handler(void)
61. {
62.     /* Go to infinite loop when Bus Fault exception occurs */
63.     while (1)
64.     {
65.     }
66. }
67.
68. /**
69.  * @brief This function handles Usage Fault exception.
70.  * @param None
71.  * @retval None
72.  */
73. void UsageFault_Handler(void)
74. {
75.     /* Go to infinite loop when Usage Fault exception occurs */
76.     while (1)
77.     {
78.     }
79. }
80.
81. /**
82.  * @brief This function handles SVCcall exception.
83.  * @param None
84.  * @retval None
85.  */
86. //void SVC_Handler(void)
87. //{
88.//}
89.
90. /**
91.  * @brief This function handles Debug Monitor exception.
92.  * @param None
93.  * @retval None
94.  */
95. void DebugMon_Handler(void)
96. {
97. }
98.
99. extern void xPortSysTickHandler(void);
100. //systick中断服务函数
101. void SysTick_Handler(void)
102. {
103.     #if (INCLUDE_xTaskGetSchedulerState == 1)
104.         if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
105.         {
106.             #endif /* INCLUDE_xTaskGetSchedulerState */
107.             xPortSysTickHandler();
108.             #if (INCLUDE_xTaskGetSchedulerState == 1)

```



```

109.     }
110.     #endif /* INCLUDE_xTaskGetSchedulerState */
111. }
112.
113. /*****
114.  * STM32F10x Peripherals Interrupt Handlers
115.  * Add here the Interrupt Handler for the used peripheral(s) (PPP), for the
116.  * available peripheral interrupt handler's name please refer to the startup
117.  * file (startup_stm32f10x_xx.s).
118.  *****/
119.
120. /**
121.  * @brief This function handles PPP interrupt request.
122.  * @param None
123.  * @retval None
124.  */
125. void PPP_IRQHandler(void)
126. {
127. }

```



11.修改main.c里面的代码

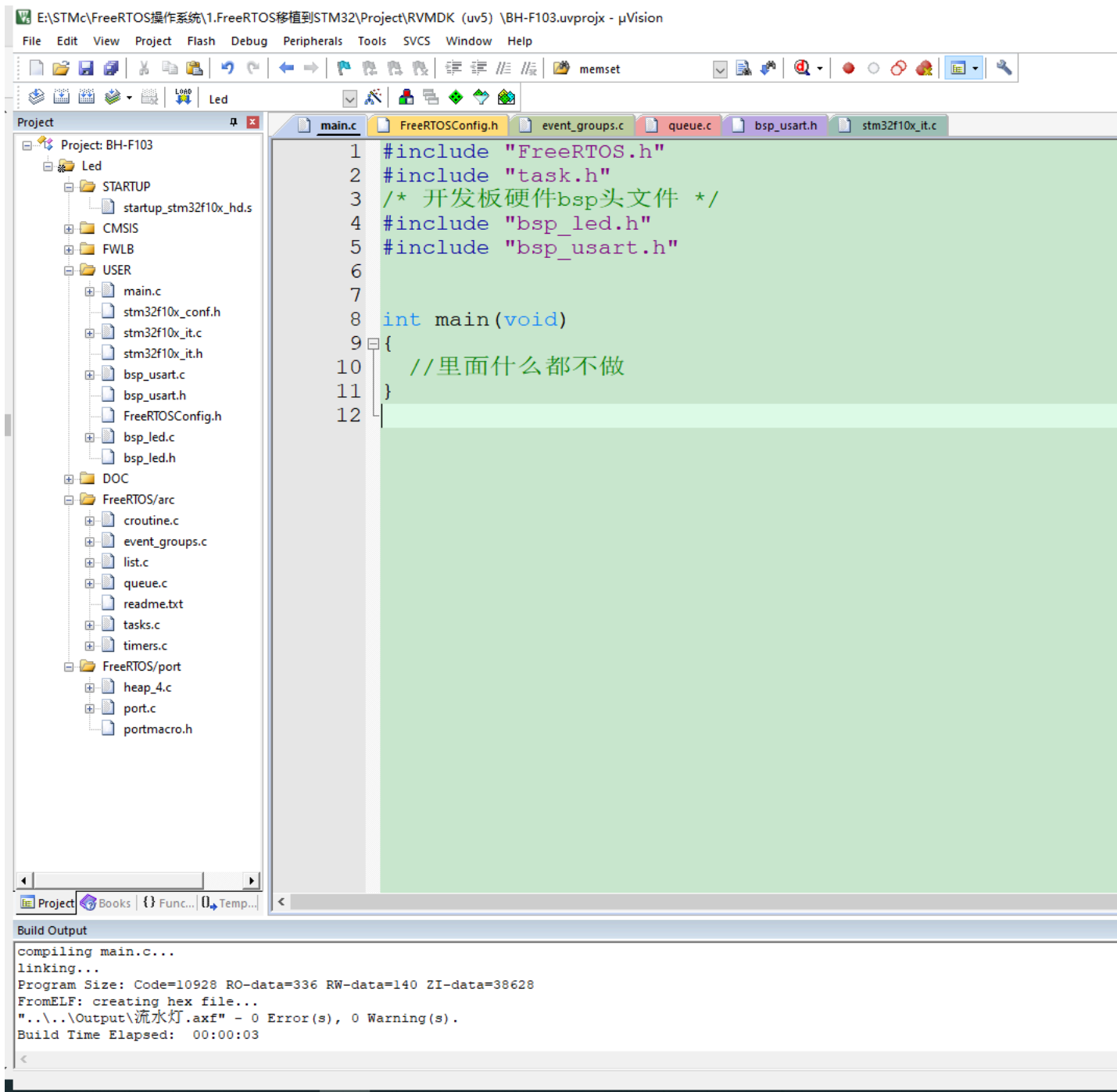
```

1. #include "FreeRTOS.h"
2. #include "task.h"
3. /* 开发板硬件bsp头文件 */
4. #include "bsp_led.h"
5. #include "bsp_usart.h"
6.
7.
8. int main(void)
9. {
10.     //里面什么都不做
11. }

```



修改好后编译看看有没有错误



笔者这是没有任何问题的。

本篇内容参考野火的Freertos内核实现与应用开发指南，详细代码可以去野火官方下载中心下载查看

<https://doc.embedfire.com/products/link/zh/latest/index.html>